-2

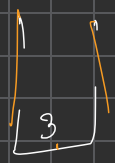
3	4	-2	5	7	-8

adyacent

negative

non-negative

0 ↓ am




```
stack<int> st

for(int i=0 ; i<n ; i++)
{
    if( st.empty())
    {
      st.push(arr[i]);
    }
    else
    {
      if ((st.top() < 0 && arr[i] >=0) ||
          (st.top()>=0 && arr[i] <0))
      {
        st.pop();
      }
      else
        st.push(arr[i];
    }
```

ab	cd	cd	ab	bc	cd	ab

ab
cd
bc

3

```
stack <string> st;
for(i=0; i<n; i++)
{
  if (st.empty())
  {
    st.push(arr[i]);
  }
  else
  {
    if (st.top() == arr[i])
      st.pop()
    else
      st.push(arr[i]);
  }

return st.size();
```

"(((" ↓ 3 insertion

↓

((()))

insertion = 1 ✓

(ii) "))(("

()()

left ↓ (⇒ open

right ↓) ⇒ c(0x)

(i) "()(()("

left = 1 × 2 = 2

right = 0

(2) ✓

"))))((("

right = 3

left = 3

0

"))(()(())"

left = 1

right = 2

3

return 0;

① "[ ] ( ) { }"

"[ ] ( ) { }"

"[( ])"

[( ]) ✗

[( )] { { [ ] } }

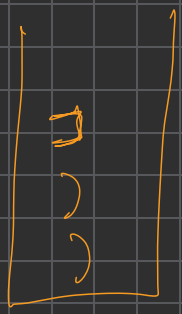
```
stack<char> st;
for(i=0; i<n; i++)
{
    if( st.empty() || str[i]=='[' || str[i]==
        '{' || str[i] == '(' )
    {
        if (str[i] == '[')
            st.push(']')
        else if (str[i] == '(')
            st.push(')')
        else if (str[i] == '{')
            st.push('}')
        else
            return 0;
    }
    else
    {
        if ( st.top() == str[i])
            st.pop();
```

else

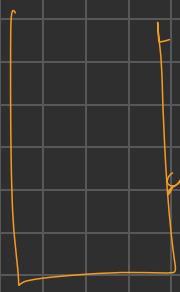
return 0;

}

"(( ] [ ]"



"( ) ( )] ( )"



return 0;

"a b # #" "c # d #"

equal hi hai

a b c # d

a b d

a b d

a b d

Yes

(1) "(aa(bdc))b(dee)"

Count = 1 2 3

1	2	2	1	3	3

(1) "(aa(bb(c(d))(f)))"

1	2	3	4	4	3	5	5	2	1